

CS6700

Reinforcement Learning (Spring 2024)

Programming Assignment 3

Semi-Markov Decision Process and Intra-Option Q-Learning

Instructor: Prof. Balaraman Ravindran

Students:

ME20B087 Janmenjaya Panda

ME20B122 Nishant Sahoo

Release Date: 06/04/2024

Submission Deadline: 20/04/2024

GitHub Repo Submission: [Link](#)



Department of Computer Science & Engineering
Indian Institute of Technology Madras

Contents

1	Environment Description	3
1.1	Taxi v3	4
1.1.1	Map	4
1.1.2	Description	4
1.1.3	Actions	5
1.1.4	Observations	5
1.1.5	Info	6
1.1.6	Rewards	6
1.1.7	Arguments	6
1.1.8	Version History	6
2	From primitive actions to closed loop policies	7
2.1	MDP Framework	7
2.2	Option	9
2.2.1	Properties of Options	9
2.3	Generalization of Bellman Equations	9
2.4	SMDP Framework	10
2.4.1	SMDP Value Learning	11
2.5	Intra-Option Framework	11
2.5.1	Theoretical Foundation	11
2.5.2	Bellman-like Equations for Options	12
2.5.3	Intuition	12
2.5.4	Intra-option Q-learning	12
2.5.5	Convergence	12
2.5.6	Advantages	13
3	Provided Options	13
3.1	Goto Red	13
3.2	Goto Green	13
3.3	Goto Yellow	14
3.4	Goto Blue	14
4	Parameters and Tuning	15
4.1	Fixed Parameters	15
4.2	Hyperparameters	15
5	Results (Provided Options)	17
5.1	SMDP	18
5.1.1	Reward Curves	18
5.1.2	A Sample Path after training	18
5.1.3	The option Q-Table	19
5.1.4	Q-Table for all the states	20
5.2	Intra-Option Q-Learning	21
5.2.1	Reward Curves	21
5.2.2	A Sample Path after training	21
5.2.3	Q-Table for Options	22

5.2.4	Q-Table for all the states	23
5.3	Inference	24
5.3.1	Reward	25
5.3.1.1	Observations	25
5.3.1.2	Possible reasons	25
5.3.2	Learnt Q Table, aka learnt policy	26
5.3.2.1	Observation	26
6	Alternate Options	27
6.1	Deterministic Options	27
6.1.1	The options	27
6.1.2	Plots	27
6.1.3	Observation and Inference	28
6.2	Goto two bottleneck cells <i>D</i> and <i>E</i>	29
6.2.1	The options	29
6.2.1.1	Goto Cell D(2, 0)	30
6.2.1.2	Goto Cell B(2, 3)	30
6.2.2	Observation and Inference	31
6.3	Goto the center C(2, 2)	31
6.3.1	Observation and Inference	33
7	Conclusion	33
8	References	33

1 Environment Description

This exercise aims to study the two variants of Hierarchical Reinforcement Learning, that are — **SMDP¹** and **Intra-Option Q-Learning**.

In this programming task, we shall utilize the [Gymnasium environments](#) for training and evaluating several policies. The following introduction of the Gymnasium has been sourced from the official documentation of the Gymnasium available at [Gymnasium](#)

Gymnasium is a maintained fork of OpenAI’s Gym library. The Gymnasium interface is simple, pythonic, and capable of representing general RL problems, and has a compatibility wrapper for old Gymnasium environments. Figure 1 shows the Lunar Lander v2 environment in the Gymnasium rendered in the human mode.

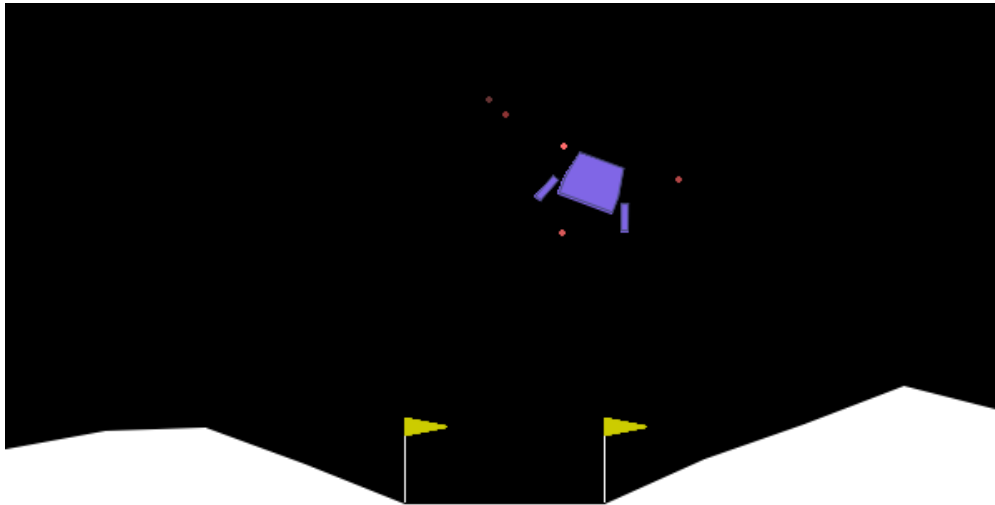


Figure 1: The [Lunar Lander v2](#) environment in the Gymnasium: an API standard for RL with a diverse collection of reference environments

```
import gymnasium as gym

env = gym.make("LunarLander-v2", render_mode="human")
observation, info = env.reset(seed=42)
for _ in range(1000):
    action = env.action_space.sample() # this is where you would insert your policy
    observation, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

¹ Semi-Markov Decision Process

We shall be dealing with the Taxi environment in the Gymnasium discussed followingly.

1.1 Taxi v3

The following description of Taxi v3 has been sourced from the official documentation of Taxi v3 available at [Taxi v3](#).

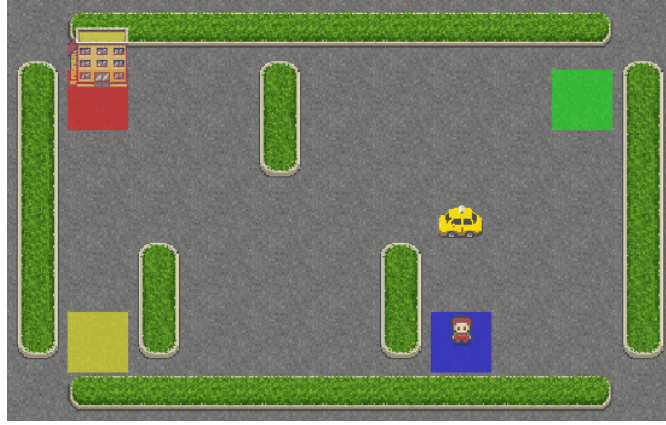


Figure 2: The [Taxi v3 environment](#) in the Gymnasium

This environment is part of the [Toy Text environments](#). Please read that page first for general information.

Variable	Value
Action Space	Discrete(6)
Observation Space	Discrete(500)
Import	<code>gym.make('Taxi-v3')</code>

Table 1: Taxi v3 specifications

The Taxi Problem from “Hierarchical Reinforcement Learning with the MAXQ Value Function Decomposition” by Tom Dietterich [1] in 1999.

1.1.1 Map

```

+-----+
|R: | : :G|
| : | : : |
| : : : : |
| | : | : |
|Y| : |B: |
+-----+

```

1.1.2 Description

There are four designated locations in the grid world indicated by R(ed), G(reen), Y(ellow), and B(lue). When the episode starts, the taxi starts off at a random square and the passenger is at a random location. The taxi drives to the passenger’s location, picks up the passenger, drives to

the passenger’s destination (another one of the four specified locations), and then drops off the passenger. Once the passenger is dropped off, the episode ends.

1.1.3 Actions

There are 6 discrete deterministic actions:

- 0: move south
- 1: move north
- 2: move east
- 3: move west
- 4: pickup passenger
- 5: drop off passenger

1.1.4 Observations

There are 500 discrete states since there are 25 taxi positions, 5 possible locations of the passenger (including the case when the passenger is in the taxi), and 4 destination locations.

Note that there are 400 states that can actually be reached during an episode. The missing states correspond to situations in which the passenger is at the same location as their destination, as this typically signals the end of an episode. Four additional states can be observed right after a successful episodes, when both the passenger and the taxi are at the destination. This gives a total of 404 reachable discrete states.

Each state space is represented by the tuple: (`taxi_row`, `taxi_col`, `passenger_location`, `destination`)

An observation is an integer that encodes the corresponding state. The state tuple can then be decoded with the “decode” method.

Passenger locations:

- 0: R(ed)
- 1: G(reen)
- 2: Y(ellow)
- 3: B(lue)
- 4: in taxi

Destinations:

- 0: R(ed)

- 1: G(green)
- 2: Y(fellow)
- 3: B(lue)

1.1.5 Info

`step` and `reset(return_info=True)` will return an info dictionary that contains “p” and “action_mask” containing the probability that the state is taken and a mask of what actions will result in a change of state to speed up training.

As Taxi’s initial state is a stochastic, the “p” key represents the probability of the transition however this value is currently bugged being 1.0, this will be fixed soon. As the steps are deterministic, “p” represents the probability of the transition which is always 1.0.

For some cases, taking an action will have no effect on the state of the agent. In v0.25.0, `info[“action_mask”]` contains a `np.ndarray` for each of the action specifying if the action will change the state.

To sample a modifying action, use `action = env.action_space.sample(info[“action_mask”])` Or with a Q-value based algorithm `action = np.argmax(q_values[obs, np.where(info[“action_mask”] == 1)[0]])`.

1.1.6 Rewards

- -1 per step unless other reward is triggered.
- +20 delivering passenger.
- -10 executing “pickup” and “drop-off” actions illegally.

1.1.7 Arguments

```
import gym
gym.make('Taxi-v3')
```

1.1.8 Version History

- v3: Map Correction + Cleaner Domain Description, v0.25.0 action masking added to the reset and step information
- v2: Disallow Taxi start location = goal location, Update Taxi observations in the rollout, Update Taxi reward threshold.
- v1: Remove (3,2) from locs, add `passidx<4` check
- v0: Initial versions release

2 From primitive actions to closed loop policies

We shall be following the paper entitled ‘Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning’ by Richard S. Sutton, Doina Precup and Satinder Singh [2] in 1999. The centre of discussion of this report is to extend the usual notion of action in this framework to include *options*, aka closed-loop policies for taking action over a period of time, for instance — opening a doorway, picking up an object, finding a switch and so on. Moreover, options enable temporally abstract knowledge and action to be included in the reinforcement learning framework in a natural and general way, in order to discover the bottleneck states or to execute a small portion of a larger task. Formally, a set of options defined over an MDP constitutes a semi-Markov decision process (SMDP), and the theory of SMDPs provides the foundation for the theory of options. We shall also explore the *intra-option* methods that are able to learn about an option from fragments of its execution. Finally, we shall establish the comparison between SMDP and intra-option processes and testify there learning efficiency as compared to only primitive action-based frameworks.

2.1 MDP Framework

To sure we all are on the same page, let us start our discussion by formally reviewing the standard reinforcement learning framework of discrete-time, finite Markov decision processes, or MDPs, which form the basis for our extension to temporally extended courses of action.

For the framework in discussion, an agent interacts with an environment at some discrete time scale, $t = 0, 1, 2, \dots$. On each time step, t , the agent perceives the state of the environment, $s_t \in \mathcal{S}$ (where $\mathcal{S}$ denote the set of states), and on that basis chooses a primitive action, $a_t \in \mathcal{A}_{s_t}$, where $\mathcal{A}_{s_t}$ denotes the sets of actions that can be taken at the state s_t . In response to each action, a_t , the environment produces one step later a numerical reward, r_{t+1} , and a next state, s_{t+1} . We let $\mathcal{A} \triangleq \bigcup_{s \in \mathcal{S}} \mathcal{A}_s$. Assuming $\mathcal{S}$ and $\mathcal{A}$, are finite, the environment’s transition dynamics can be modeled by one-step state-transition probabilities from state s to state s' through action a at time step t ,

$$p(s_{t+1}|s_t, a_t) := \mathbb{P}(s_{t+1} = s' | s_t = s, a_t = a)$$

and one step expected reward,

$$r(s_t, a_t) := \mathbb{E}[(r_{t+1} | s_t = s, a_t = a)]$$

for all $s, s' \in \mathcal{S}$ and $a \in \mathcal{A}_s$. These two sets of quantities together constitute the one-step model of the environment.

The agent’s objective is to learn a *Markov policy*, a mapping from states to probabilities of taking each available primitive action, $\pi : \mathcal{S} \times \mathcal{A} \leftarrow [0, 1]$, that maximizes the expected return (defined as the cumulative discounted reward) from each state s :

$$\begin{aligned} V_\pi(s) &:= \mathbb{E}_\pi [r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots | s_t = s] \\ &= \mathbb{E}_\pi [r_{t+1} + \gamma V_\pi(s_{t+1}) | s_t = s] \\ &= \sum_{a \in \mathcal{A}_s} \pi(s, a) \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) V_\pi(s') \right] \end{aligned} \tag{1}$$

where $\pi(s, a)$ is the probability with which the policy π chooses action $a \in \mathcal{A}_s$ in state s , and $\gamma \in [0, 1]$ is the *discount factor*. The quantity, $V_\pi(s)$, is called the *value* of state s under policy

π , and V_π is called the *state-value* function for the policy π . We let Π denote the universe of all policies that can be followed by the agent in the environment.

A policy μ is called an *optimal policy* if $V_\mu(s) \geq V_\pi(s)$ for each $(s, \pi) \in \mathcal{S} \times \Pi$. The reader may verify that for any two optimal policies μ and ν , it holds that $V_\mu(s) = V_\nu(s)$ for each state s . The *optimal state-value function*, denoted as V_* gives the value of each state under an optimal policy:

$$\begin{aligned} V_*(s) &:= \max_{\pi} V_\pi(s) \\ &= \max_{\pi} \mathbb{E}_{\pi} [r_{t+1} + \gamma V_*(s+1) | s_t = s] \\ &= \max_{a \in \mathcal{A}_s} \mathbb{E} [r_{t+1} + \gamma V_*(s+1) | s_t = s, a_t = a] \\ &= \max_{a \in \mathcal{A}_s} \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) V_*(s') \right] \end{aligned} \tag{2}$$

A policy π is an optimal policy if and only if $V_\pi(s) = V_*(s)$ for each state s . Clearly, given V_* , we may easily construct an optimal policy π as follows:

$$\pi(s, a) := \arg \max_{a \in \mathcal{A}_s} \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) V_*(s') \right] \quad \forall (s, a) \in \mathcal{S} \times \mathcal{A}$$

If we treat the values, $V_\pi(s)$ or $V_*(s)$, as unknowns, then a set of Bellman equations, for all $s \in \mathcal{S}$, forms a system of equations whose unique solution is in fact V_π or V_* as discussed earlier. This fact is key to the way in which all temporal-difference and dynamic programming methods estimate value functions.

There are similar value functions and Bellman equations for state-action pairs, rather than for states, which are particularly important for learning methods. The value of taking action a in state s under policy π , denoted $Q_\pi(s, a)$, is the expected discounted future reward starting in s , taking a , and henceforth following π :

$$\begin{aligned} Q_\pi(s, a) &:= \mathbb{E}_{\pi} [r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots | s_t = s, a_t = \pi(s_t)] \\ &= \mathbb{E}_{\pi} [r_{t+1} + \gamma V_\pi(s+1) | s_t = s, a_t = \pi(s_t)] \\ &= r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) V_\pi(s') \end{aligned} \tag{3}$$

This is known as the action-value function for policy π . The optimal action-value function is given by:

$$\begin{aligned} Q_*(s, a) &:= \max_{\pi} Q_\pi(s, a) \\ &= \max_{\pi} \mathbb{E}_{\pi} [r_{t+1} + \gamma V_*(s+1) | s_t = s, a_t = \pi(s_t)] \\ &= \max_{a \in \mathcal{A}_s} \mathbb{E} [r_{t+1} + \gamma V_*(s+1) | s_t = s, a_t = a] \\ &= \max_{a \in \mathcal{A}_s} \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, a) V_*(s') \right] \end{aligned} \tag{4}$$

Finally, many tasks are episodic in nature, involving repeated trials, or episodes, each ending with a reset to a standard state or state distribution. Episodic tasks include a special terminal state; arriving in this state terminates the current episode. In an episodic task, values are defined by the expected cumulative reward up until termination rather than over the infinite future (or, equivalently, we can consider the terminal state to transition to itself forever with a reward of zero).

2.2 Option

The term *options* is used for our generalization of primitive actions to include temporally extended courses of action. Options are a form of temporal abstraction that allows agents to plan and act at multiple levels of granularity. They extend the standard reinforcement learning framework by introducing a set of high-level actions, each of which is a closed-loop policy for taking action over a period of time. Formally, an option o is defined as a triple $o = \langle I_o, \pi_o, \beta_o \rangle$, where:

- $I_o \subseteq S$ is the initiation set, indicating the states from which the option can be initiated.
- $\pi_o : S \times A \rightarrow [0, 1]$ is the policy, which defines the behaviour of the option.
- $\beta_o : S \rightarrow [0, 1]$ is the termination condition, which specifies the probability of the option terminating in each state.

If the option is taken, then actions are selected according to π until the option terminates stochastically according to β . In particular, a Markov option executes as follows. First, the next action a_t is selected according to probability distribution $\pi(s_t, \cdot)$. The environment then makes a transition to state s_{t+1} , where the option either terminates, with probability $\beta(s_{t+1})$, or else continues, determining a_{t+1} according to $\pi(s_{t+1}, \cdot)$, possibly terminating in s_{t+2} according to $\beta(s_{t+2})$, and so on. When the option terminates, the agent has the opportunity to select another option. We shall denote the set of options that are executable from state s by $\mathcal{O}_s$ and the set of options is defined as $\mathcal{O} := \bigcup_{s \in S} \mathcal{O}_s$.

2.2.1 Properties of Options

Options have several key properties:

- **Markov Property:** An option is Markov if its behavior and termination depend only on the current state and not on the history.
- **Semi-Markov Property:** An option is semi-Markov if its duration can vary and depends on the state in which it was initiated.
- **Compositionality:** Options can be composed to form new options, allowing for hierarchical decision-making.

2.3 Generalization of Bellman Equations

Planning with options requires a model of their consequences. Fortunately, the appropriate form of a model for options, analogous to the $r(s, a)$ and $p(s'|s, a)$ defined earlier for actions, is known from existing SMDP theory. For each state in which an option may be started, this kind of model predicts the state in which the option will terminate and the total reward received along the way. These quantities are discounted in a particular way. For any option o , let $\mathcal{E}(o, s, t)$ denote the event of o being initiated in state s at time t . Then the reward part of the model of o for any state $s \in S$ is:

$$r(s, o) = \mathbb{E}[r_{t+1} + \gamma r_{t+2} + \dots \gamma^{k-1} r_{t+k} | \mathcal{E}(o, s, t)] \quad (5)$$

where $t + k$ is the time at which o terminates. The state-prediction part of the model of o for state s is:

$$p(s'|s, o) = \sum_{k=1}^{\infty} p(s', k) \gamma^k \quad (6)$$

for all $s' \in \mathcal{S}$, where $p(s', k)$ is the probability that the option terminates in s' after k steps starting from the state s .

There are generalizations of optimal value functions and optimal Bellman equations to options and to policies over options. Of course, the conventional optimal value functions V_* and Q_* are not affected by the introduction of options; one can ultimately do just as well with primitive actions as one can with options. Nevertheless, it is interesting to know how well one can do with a restricted set of options that does not include all the primitive actions. For example, in planning one might first consider only high-level options in order to find an approximate plan quickly. Let us denote the set of all options by $\mathcal{O}$ and the set of all policies selecting only from options in $\mathcal{O}$ by $\Pi(\mathcal{O})$. Then the optimal value function given that we can select actions only from $\mathcal{O}$ is:

$$\begin{aligned} V_*^{\mathcal{O}}(s) &:= \max_{\mu \in \Pi(\mathcal{O})} V_{\mu}(s) \\ &= \max_{o \in \mathcal{O}_s} \mathbb{E}_{\mu} [r_{t+1} + \dots + \gamma^{k-1} r_{t+k} + \gamma^k V_*^{\mathcal{O}}(s+k) | \mathcal{E}(o, s, t)] \\ &= \max_{o \in \mathcal{O}_s} \left[r(s, o) + \gamma^k \sum_{s' \in \mathcal{S}} p(s'|s, o) V_*^{\mathcal{O}}(s') \right] \\ &= \max_{o \in \mathcal{O}_s} \mathbb{E} [r + \gamma^k V_*^{\mathcal{O}}(s') | \mathcal{E}(o, s)] \end{aligned} \quad (7)$$

Here k is the duration of o when taken in s and $\mathcal{E}(o, s)$ denotes the option o being initiated in state s . Likewise, the value function and Bellman equations for optimal option values are:

$$\begin{aligned} Q_*^{\mathcal{O}}(s, o) &:= \max_{\mu \in \Pi(\mathcal{O})} Q_{\mu}(s, o) \\ &= \mathbb{E}_{\pi} [r_{t+1} + \dots + \gamma^{k-1} r_{t+k} + \gamma^k V_*^{\mathcal{O}}(s+k) | \mathcal{E}(o, s, t)] \\ &= r(s, o) + \gamma^k \sum_{s' \in \mathcal{S}} p(s'|s, o) V_*^{\mathcal{O}}(s') \\ &= \mathbb{E} [r + \gamma^k V_*^{\mathcal{O}}(s') | \mathcal{E}(o, s)] \end{aligned} \quad (8)$$

Options enable agents to make decisions that span variable lengths of time, simplifying the decision-making process by focusing on higher-level goals. The use of options can lead to more efficient learning and planning, as they allow the agent to consider extended courses of action rather than only immediate one-step actions.

2.4 SMDP Framework

Semi-Markov Decision Processes (SMDPs) are an extension of Markov Decision Processes (MDPs) that allow for actions to have variable durations. SMDPs are particularly useful for modeling decision-making problems where actions take place in continuous time or have delays associated with them. Options are closely related to the actions in a special kind of decision problem known as a semi-Markov decision process, or SMDP. In fact, any MDP with a fixed set of options is an SMDP. The relationship among MDPs, options, and SMDPs provides a basis for the theory of planning and learning methods with options. However, this theory is limited due to its treatment

of options as indivisible units without internal structure.

2.4.1 SMDP Value Learning

The problem of finding an optimal policy over a set of options O can also be addressed by learning methods. Because the MDP augmented by the options is an SMDP, we can apply SMDP learning methods. Each option is viewed as an indivisible, opaque unit. When the execution of option o is started in state s , we next jump to the state s' in which o terminates. Based on this experience, an approximate option-value function $Q(s, o)$ is updated.

The SMDP version of one-step Q-learning, which we call **SMDP Q-learning**, updates after each option termination by:

$$Q(s, o) \leftarrow Q(s, o) + \alpha \left[r + \gamma^k \max_{o' \in \mathcal{O}_s} Q(s', o') - Q(s, o) \right], \quad (9)$$

where s is the step where the agent started executing the option o , the state s' is the state where the option terminated, and k denotes the number of time steps elapsing between s and s' . Here r denotes the cumulative discounted reward over this time, that is:

$$r = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{k-1} r_{t+k}$$

and α is the step-size parameter. The estimate $Q(s, o)$ converges to $Q_o^*(s, o)$ for all $s \in S$ and $o \in O$ under conditions similar to those for conventional Q-learning, from which it is easy to determine an optimal policy.

2.5 Intra-Option Framework

Intra-option model learning is a method for learning the model, $r_o(s)$ and $p_o(s'|s)$, of an option o , given experience and knowledge of o (i.e., of its initiation set I , policy π , and termination condition β). This method requires that π be deterministic and that the option be Markov. The general approach for a semi-Markov option is to execute the option to termination multiple times in each state s , recording the resultant next state s' , cumulative discounted reward r , and elapsed time k .

For Markov options, special temporal-difference methods can be used to learn about the model of an option before the option terminates. These are called intra-option methods because they learn about an option from a fragment of experience "within" the option. Intra-option methods can even be used to learn about an option without ever executing it, as long as some selections are made that are consistent with the option. These methods are examples of off-policy learning because they learn about the consequences of one policy while actually behaving according to another.

Intra-option value learning is a method for learning the value of options, which are temporally extended courses of action, within the framework of Markov Decision Processes (MDPs). This method is particularly efficient because it allows for learning from experience that is only partially related to the option being evaluated.

2.5.1 Theoretical Foundation

The value of taking an option o in state s under policy π , denoted $Q^\pi(s, o)$, is the expected discounted future reward starting in s , taking o , and henceforth following π . For Markov options, we can consider intra-option methods that learn about the value of an option before it terminates.

2.5.2 Bellman-like Equations for Options

For a Markov option $o = (I, \pi, \beta)$, where I is the initiation set, π is the policy, and β is the termination condition, the Bellman-like equation relating $Q^*(s, o)$ to expected values of $U^*(s', o)$, where s' is the immediate successor to s after initiating o , is given by:

$$Q^*(s, o) = \sum_{a \in A_s} \pi(s, a) [r + \gamma U^*(s', o)] \quad (10)$$

where r is the immediate reward upon arrival in s' .

2.5.3 Intuition

We are learning to approximate $Q^*(s, o)$ and that o is Markov. Based on an execution of o from t to $t + k$, SMDP methods extract a single training example for $Q^*(s, o)$. But because o is Markov, it is, in a sense, also initiated at each of the steps between t and $t + k$. The jumps from each intermediate s_i to s_{t+k} are also valid experiences with o , experiences that can be used to improve estimates of $Q^*(s_i, o)$. Or consider an option that is very similar to o and which would have selected the same actions, but which would have terminated one step later, at $t + k + 1$ rather than at $t + k$. Formally this is a different option, and formally it was not executed, yet all this experience could be used for learning relevant to it. In fact, an option can often learn something from experience that is only slightly related (occasionally selecting the same actions) to what would be generated by executing the option. This is the idea of off-policy training—to make full use of whatever experience occurs to learn as much as possible about all options irrespective of their role in generating the experience. To make the best use of experience we would like off-policy and intra-option versions of value-learning methods such as Q-learning. It is convenient to introduce new notation for the value of a state–option pair given that the option is Markov and executing upon arrival in the state:

$$U_o^*(s, o) = (1 - \beta(s)) Q_o^*(s, o) + \beta(s) \max_{o' \in O} Q_o^*(s, o') \quad (11)$$

$$Q_o^*(s, o) = \sum_{a \in A_s} \pi(s, a) \left[r_a^s + \sum_{s'} p(s'|s, a) U_o^*(s', o) \right] \quad (12)$$

2.5.4 Intra-option Q-learning

Intra-option Q-learning updates the value of an option o based on the Bellman-like equation. Suppose action a_t is taken in state s_t to produce next state s_{t+1} and reward r_{t+1} , and that a_t was selected according to the distribution $\pi(s_t, \cdot)$. The update rule is:

$$Q(s_t, o) \leftarrow Q(s_t, o) + \alpha [r_{t+1} + \gamma U(s_{t+1}, o) - Q(s_t, o)] \quad (13)$$

where $U(s, o)$ is defined as:

$$U(s, o) = (1 - \beta(s)) Q(s, o) + \beta(s) \max_{o' \in O} Q(s, o') \quad (14)$$

2.5.5 Convergence

Under certain conditions, such as every action being executed in every state infinitely often, intra-option Q-learning converges with probability 1 to the optimal Q-values, Q_o^* , for every option.

2.5.6 Advantages

Intra-option methods are advantageous because they allow for learning about the value of an option from fragments of experience within the option. They can even be used to learn about an option without ever executing it, as long as some selections are made that are consistent with the option.

3 Provided Options

In this section, we describe the provided options, that are implemented. Observe that the Goal cell/ termination cell lies always in one of the Red, Green, Yellow and Blue cells. Likewise, the position of the passenger, apart from being in the taxi, always is at one of the coloured cells. Thus, we have the following four different (intuitive) options as described below:

3.1 Goto Red

The ‘Goto Red’ option $o_R := \langle I_R, \pi_R, \beta_R \rangle$ is defined as follows:

- $I_R :=$ The set of all the states in `env.observation_space` where the taxi is not located at the Red cell.
- $\pi_R :=$ Choose among the first four primitive actions, that are — move south, move north, move east, move east ϵ -greedily. The reader may recall the ϵ -greedy policy:

$$\pi_R(s, a) = \begin{cases} 1 - \epsilon + \frac{1}{4}\epsilon & \text{if } a = \arg \max_{a \in \mathcal{A}_s} Q_\pi(s, a) \\ \frac{\epsilon}{4} & \text{if } a \in \{0, 1, 2, 4\} \text{ and } a \neq \arg \max_{a \in \mathcal{A}_s} Q_\pi(s, a) \\ 0 & \text{otherwise} \end{cases}$$

- For each state s , the ‘Goto Red’ termination probability goes as follows:

$$\beta_R(s) = \begin{cases} 1 & \text{if at state } s \text{ taxi is at the Red cell} \\ 0 & \text{otherwise} \end{cases}$$

3.2 Goto Green

The ‘Goto Red’ option $o_G := \langle I_G, \pi_G, \beta_G \rangle$ is defined as follows:

- $I_G :=$ The set of all the states in `env.observation_space` where the taxi is not located at the Green cell.
- $\pi_G :=$ Choose among the first four primitive actions, that are — move south, move north, move east, move east ϵ -greedily.
- For each state s , the ‘Goto Green’ termination probability goes as follows:

$$\beta_G(s) = \begin{cases} 1 & \text{if at state } s \text{ taxi is at the Green cell} \\ 0 & \text{otherwise} \end{cases}$$

3.3 Goto Yellow

The ‘Goto Yellow’ option $\mathbf{o}_Y := \langle I_Y, \pi_Y, \beta_Y \rangle$ is defined as follows:

- $I_Y :=$ The set of all the states in `env.observation_space` where the taxi is not located at the Yellow cell.
- $\pi_Y :=$ Choose among the first four primitive actions, that are — move south, move north, move east, move east ϵ -greedily.
- For each state s , the ‘Goto Yellow’ termination probability goes as follows:

$$\beta_Y(s) = \begin{cases} 1 & \text{if at state } s \text{ taxi is at the Yellow cell} \\ 0 & \text{otherwise} \end{cases}$$

Analogously, we defined the ‘Goto Blue’ option, denoted as $\mathbf{o}_B$.

3.4 Goto Blue

The ‘Goto Blue’ option $\mathbf{o}_B := \langle I_B, \pi_B, \beta_B \rangle$ is defined as follows:

- $I_B :=$ The set of all the states in `env.observation_space` where the taxi is not located at the Blue cell.
- $\pi_B :=$ Choose among the first four primitive actions, that are — move south, move north, move east, move east ϵ -greedily.
- For each state s , the ‘Goto Blue’ termination probability goes as follows:

$$\beta_B(s) = \begin{cases} 1 & \text{if at state } s \text{ taxi is at the Blue cell} \\ 0 & \text{otherwise} \end{cases}$$

4 Parameters and Tuning

In this section, we talk about the hyperparameter tuning and the value assigned to other parameters.

4.1 Fixed Parameters

Firstly, γ , as per the problem definition is set to 1. For ϵ -greedy we follow a decayed value of ϵ over steps. We fix the ϵ parameters for each of the experiments. Moreover, the following table captures the values of parameters that are fixed throughout our journey.

Parameter	Value
Gamma (γ)	0.9
Epsilon (ϵ_t)	$\max(0.01 * 0.99^t, 10^{-5})$
Number of Episodes	5000
Max number of Steps	1000
Number of runs	5
Moving average window size	10

Table 2: Fixed Parameters

The reader may note that the rate of change in convergence is negligible for any result after 5000 episodes. Since we have only 25 cells (at most once 50 cell visits; in 25 of which the passenger is inside the taxi and in the rest passenger is outside), it is reasonable to choose the maximum number of steps till the goal is reached to be 1000. Note that this is devoid of the maximum number of steps taken inside any option (which can be as large as defined by `gymnasium` environment). The number of runs over which it is averaged is set to 5 as usual.

The reward plots are stochastic in nature. So it is helpful to consider the plots with a moving average window to get a better insight of how it is changing over the episodes. The moving average window size is set to 15.

4.2 Hyperparameters

As discussed above, everything boils down to the learning rate (α) which is our only hyperparameter. The choice of this learning rate plays a crucial role in the performance and convergence properties of temporal difference (TD) learning algorithms. Both too small and too large values of α can lead to suboptimal learning behaviour or even failure to converge. Here's why:

- **Small Alpha (α):**
 - *Slow Learning:* When α is too small, the agent's learning process becomes slow because it updates its estimates only by a small fraction of the difference between the old estimate and the target estimate. This slow learning can make it take a long time for the agent to converge to optimal policies, especially in large state spaces.
 - *Over-Adaptation:* Conversely, if the learning rate is too small, the agent may not adapt quickly enough to changes in the environment. This can result in the agent

being unable to capture important changes in the environment dynamics, leading to suboptimal policies.

- **Large Alpha (α):**

- *Instability:* When α is too large, the learning process becomes unstable. The agent’s estimates may oscillate or diverge instead of converging to optimal values. This instability arises because large learning rates can cause the agent to overreact to noisy or misleading observations, leading to erratic behaviour.
- *Convergence Issues:* High learning rates may cause the agent’s estimates to overshoot the true values, preventing convergence to the optimal policy. Additionally, large updates can overwrite valuable information learned in previous iterations, hindering the agent’s ability to converge.

Choosing an appropriate learning rate α involves striking a balance between exploration (updating estimates based on new experiences) and exploitation (leveraging learned knowledge). For tuning α , we consider several values of candidate α s, and picked the one which generated **maximum average reward over all the episodes**. This definition of tuning is reasonable, as it captures how quickly a particular policy/ strategy converged and if at all it converged to which reward it converged. The average reward is directly proportional to each of those.

Followingly, we show the average reward over all the episodes obtained (by averaging it over five runs) for different values of α for both SMDP and Intra-option learning (Primitive actions + Options). For the following plot at the RHS, X-axis refers to the episode and Y-axis captures the moving average reward/

Learning Rate (α)	Average Rewards
0.01	-117.09322
0.05	-91.57795
0.1	-51.48118
0.2	-31.80521
0.5	-19.13376

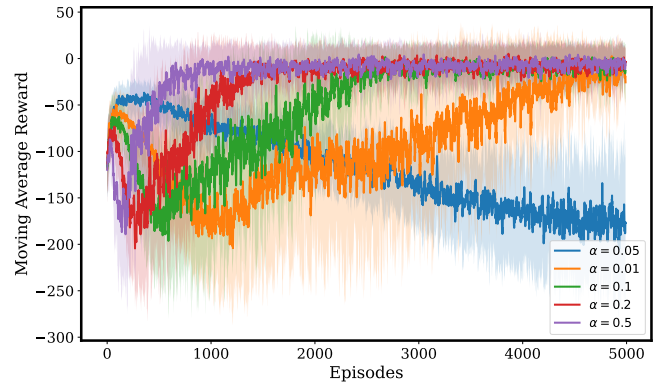


Figure 3: Tuning α in SMDP (Primitive actions + Options)

Learning Rate (α)	Average Rewards
0.01	-154.81423
0.05	-34.13420
0.1	-14.593791
0.2	-4.88232
0.5	0.745475

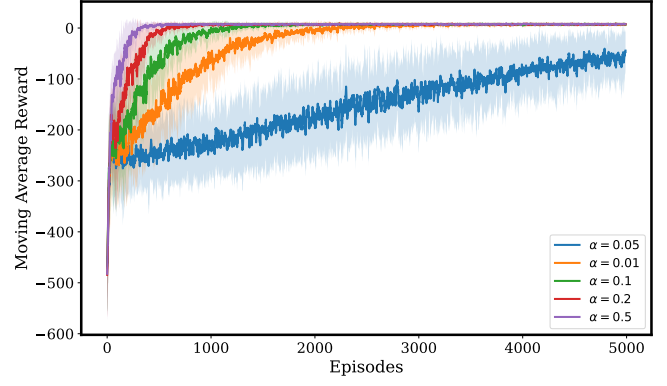


Figure 4: Tuning α in Intra Option (Primitive actions + Options)

Clearly, both of them performed the best for $\alpha = 0.5$. Henceforth, we set α to be 0.5 for the rest of the experiment.

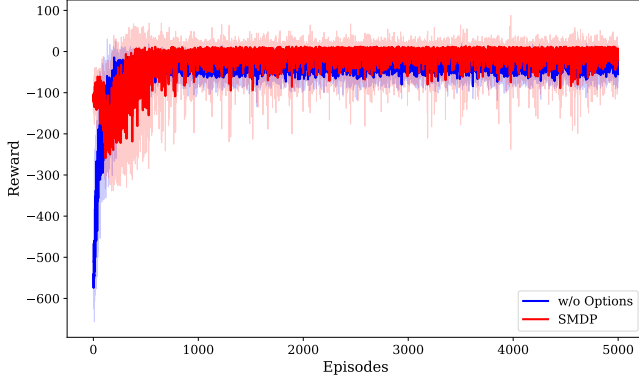
5 Results (Provided Options)

In this section, we plot the required results concerning the SMDP and the Intra-option framework.

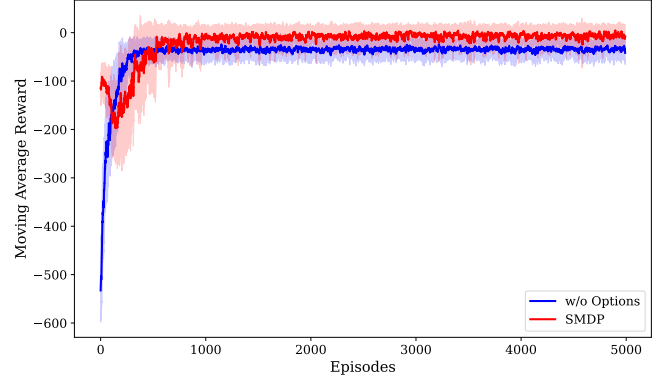
5.1 SMDP

The following plots capture the required result for SMDP (compared with the conventional Q-Learning without options).

5.1.1 Reward Curves



(a) Averaged over 5 runs
(No moving window averaging)

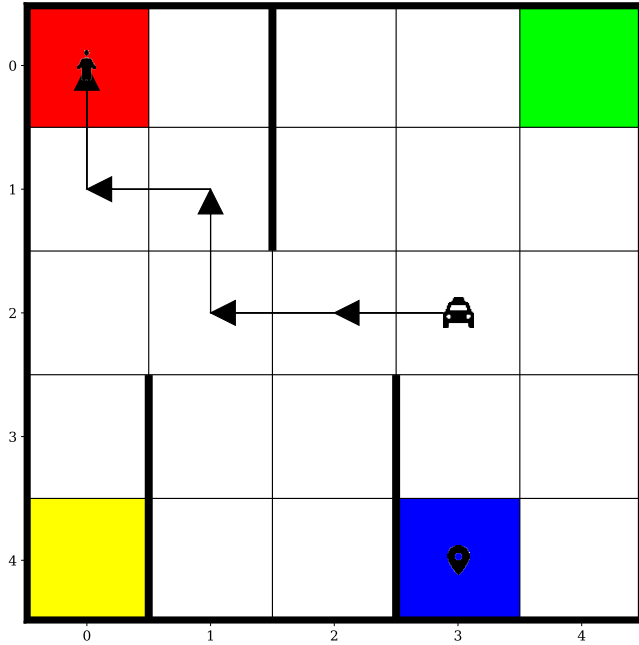


(b) veraged over 5 runs
(Moving window averaging of size 10)

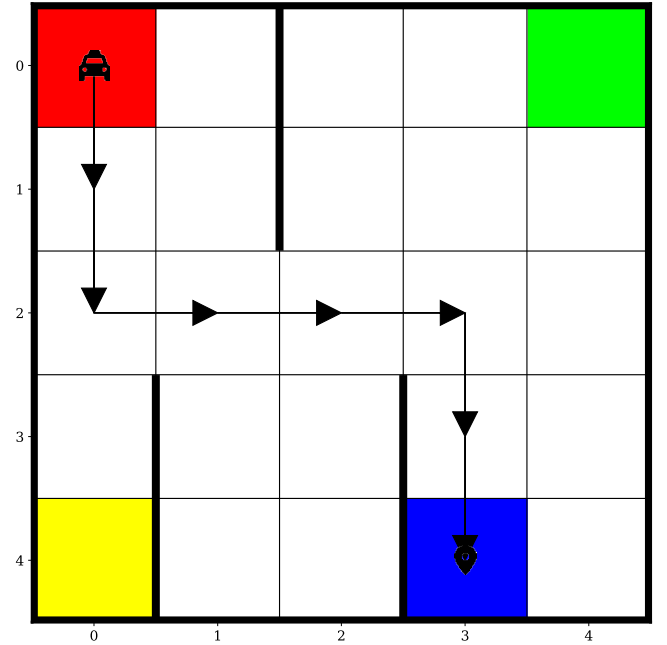
Figure 5: Reward vs Episode: SMDP vs Conventional Q-Learning (w/o options)

5.1.2 A Sample Path after training

An analogous post-training simulation (concerning the SMDP) to the following can be found [here](#).



(a) From start to pick up

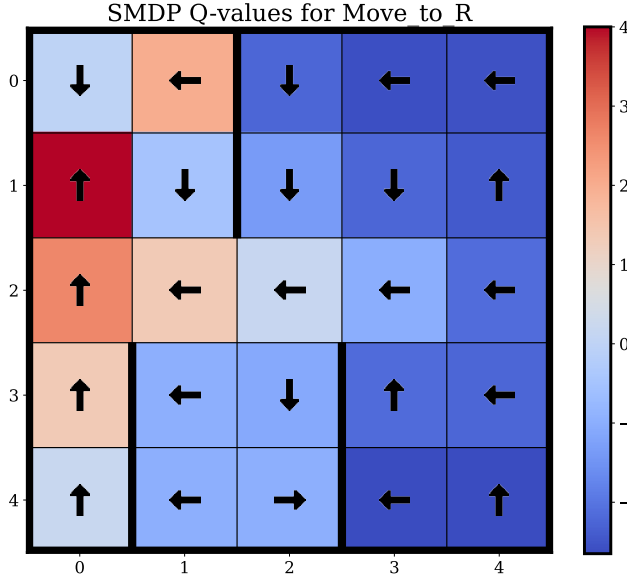


(b) From pick up to drop off

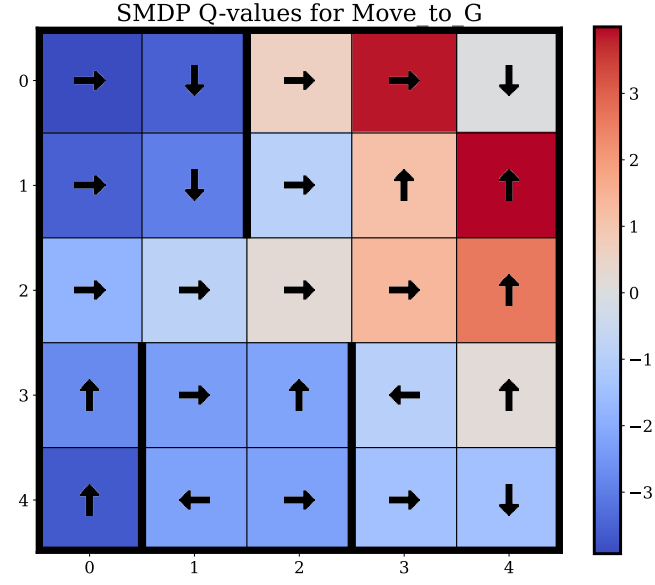
Figure 6: A sample path taken by SMDP after training

5.1.3 The option Q-Table

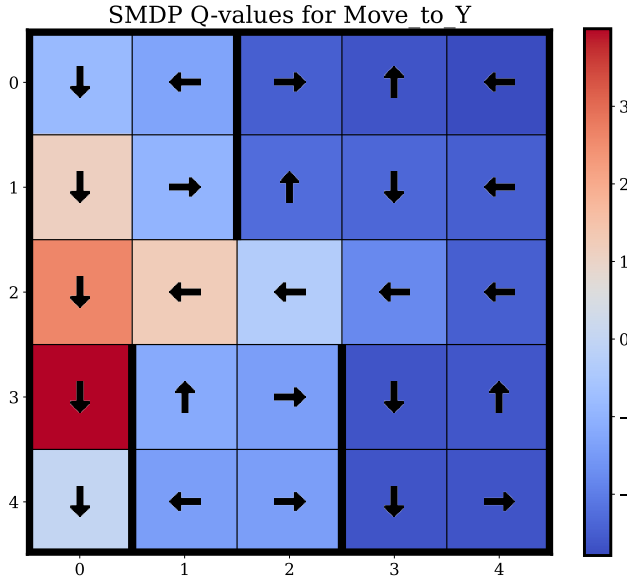
The following plots show the option Q-table for each of the four options, ‘Goto Red’, ‘Goto Green’, ‘Goto Yellow’ and ‘Goto Blue’ after training under SMDP framework.



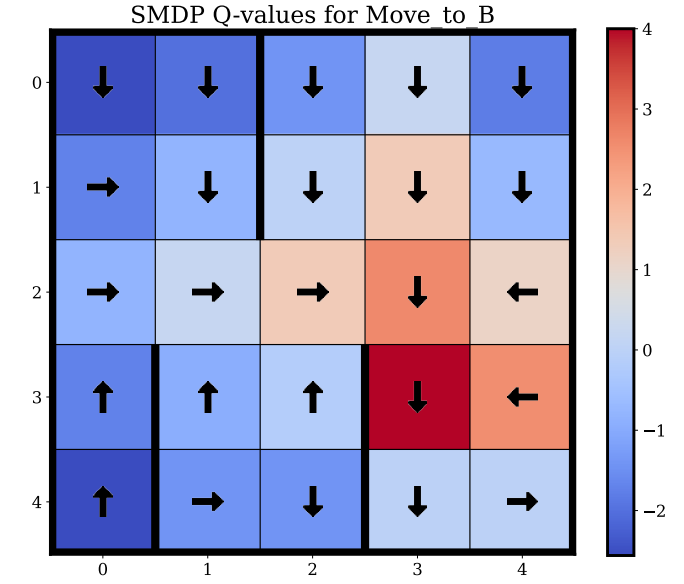
(a) Option Q-table for ‘Goto Red’



(b) Option Q-table for ‘Goto Green’



(c) Option Q-table for ‘Goto Yellow’



(d) Option Q-table for ‘Goto Blue’

Figure 7: Option Q-table for each of the provided options learnt with SMDP Framework

5.1.4 Q-Table for all the states

The following plot captures the (SMDP Global) Q-table over all primitive actions and options for all five distinct passenger locations and all four distinct destination locations.

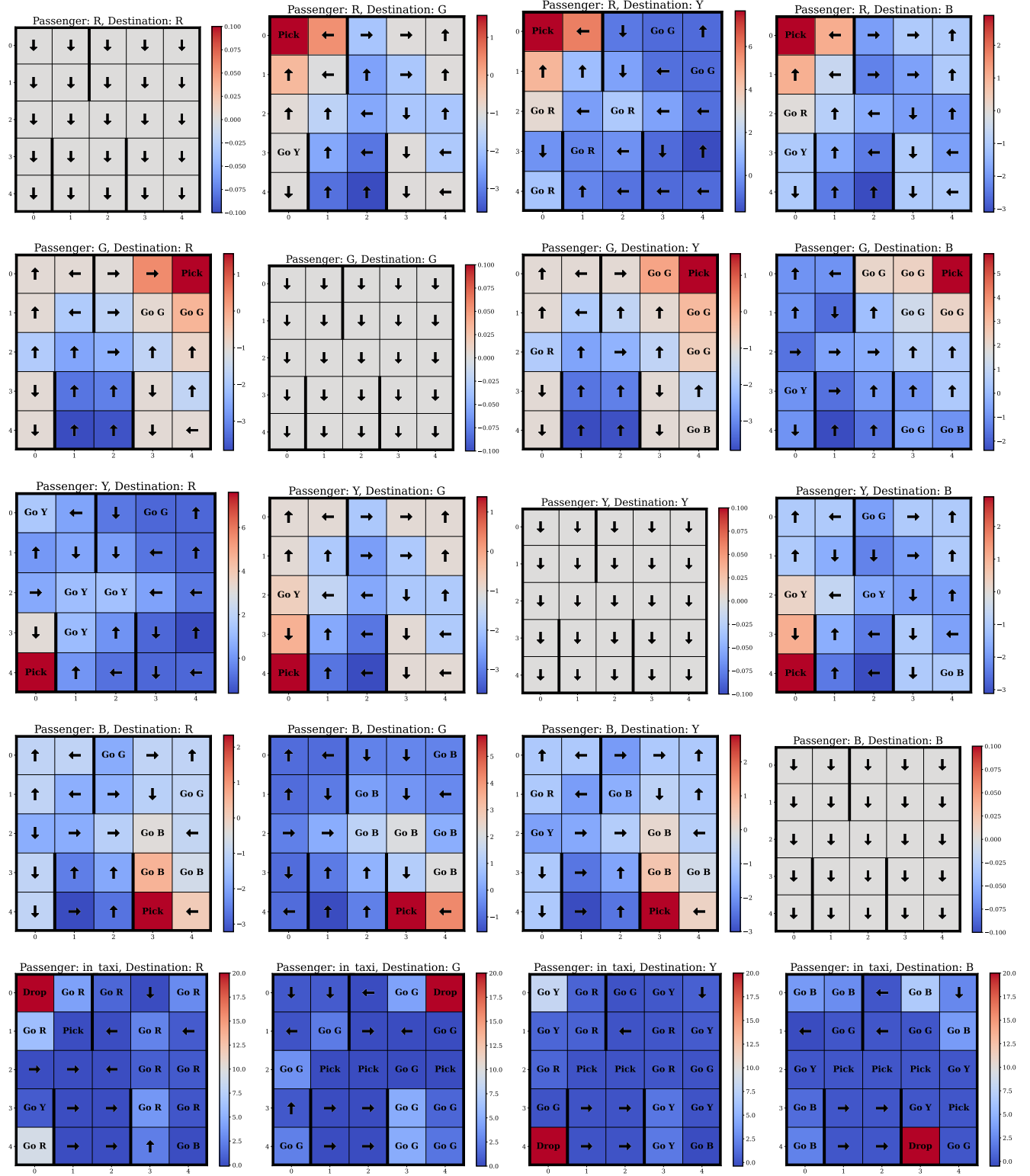
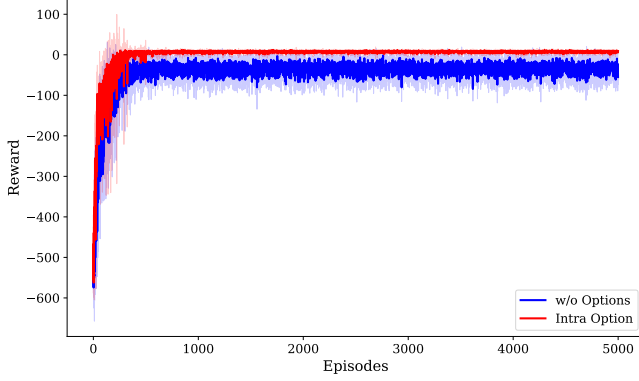


Figure 8: The Q-Table learnt through SMDP for 20 distinct environment scenarios

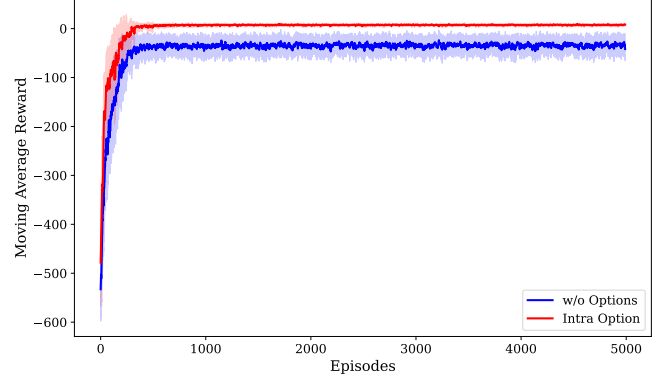
5.2 Intra-Option Q-Learning

The following plots capture the required result for Intra-option Q-Learning (compared with conventional Q-Learning without options).

5.2.1 Reward Curves



(a) Averaged over 5 runs
(No moving window averaging)

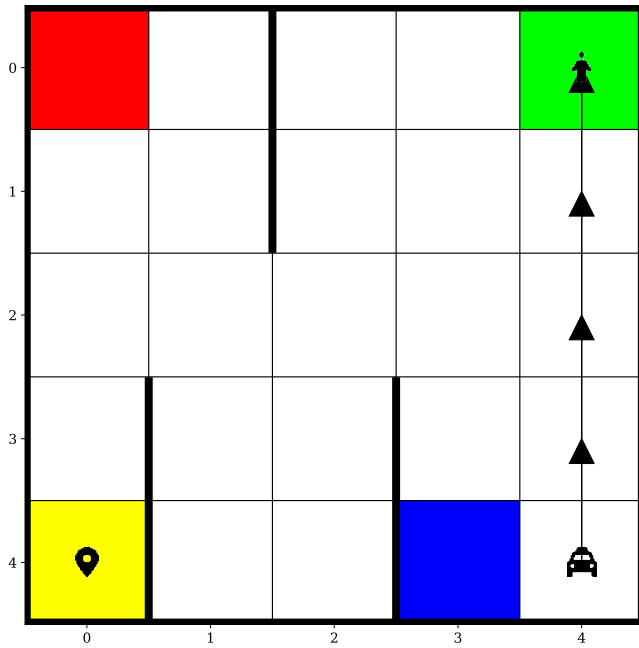


(b) Averaged over 5 runs
(Moving window averaging of size 10)

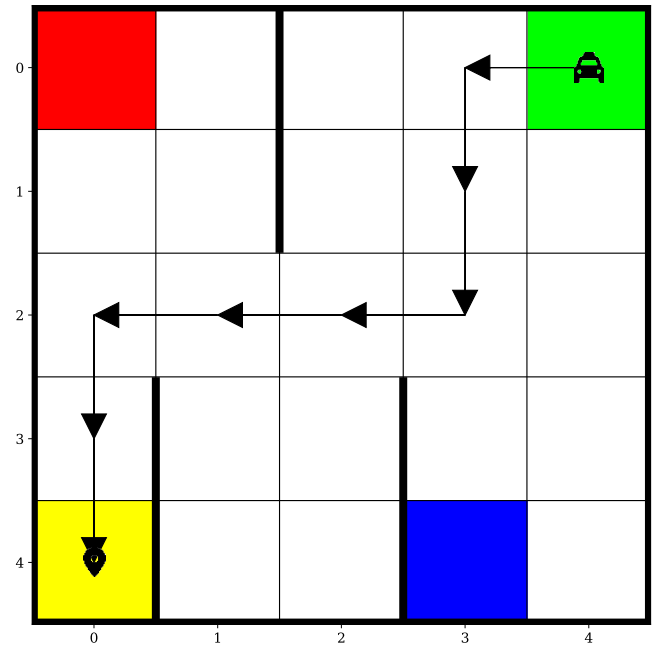
Figure 9: Reward vs Episode: Intra-option vs Conventional Q-Learning (w/o options)

5.2.2 A Sample Path after training

An analogous post-training simulation (concerning the Intra-option) to the following can be found [here](#).



(a) From start to pick up

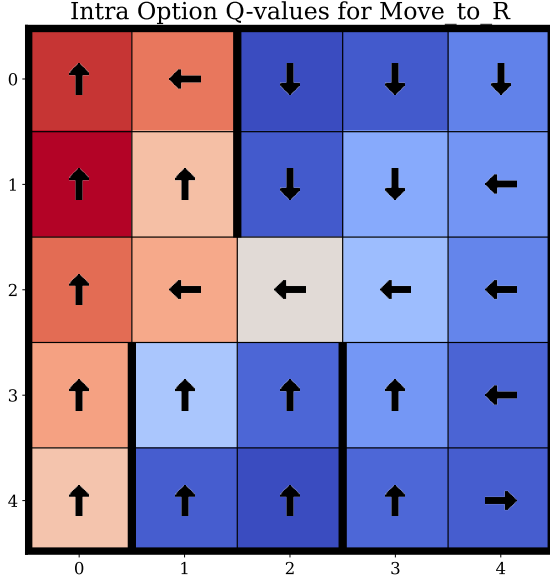


(b) From pick up to drop off

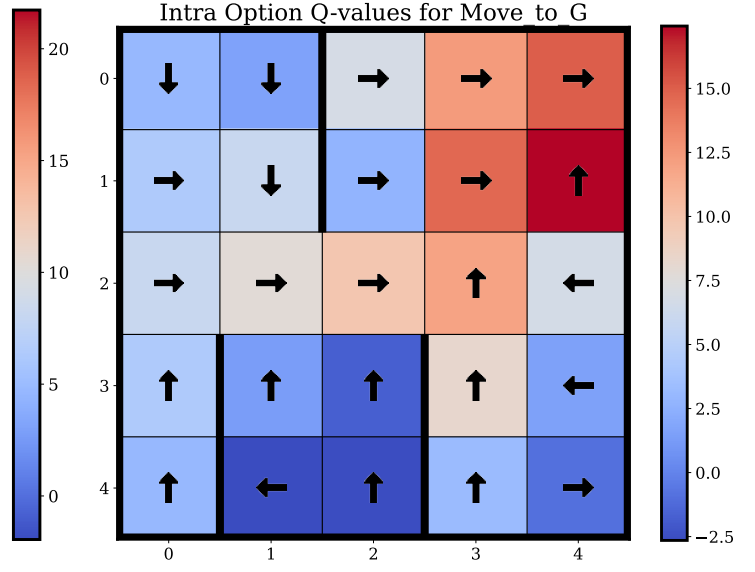
Figure 10: A sample path taken by Intra-option after training

5.2.3 Q-Table for Options

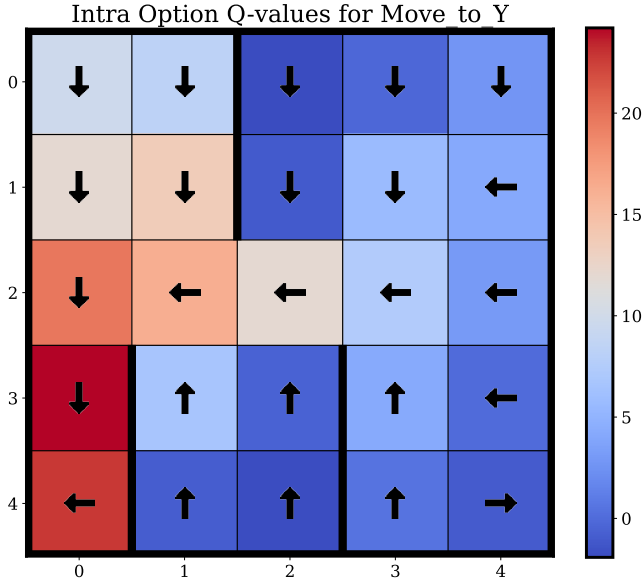
The following plots show the option Q -table for each of the four options, ‘Goto Red’, ‘Goto Green’, ‘Goto Yellow’ and ‘Goto Blue’ after training under Intra option framework.



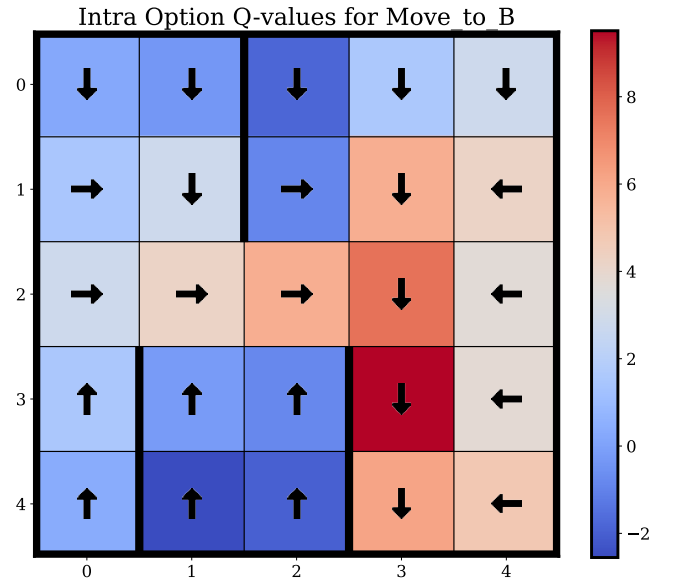
(a) Option Q -table for ‘Goto Red’



(b) Option Q -table for ‘Goto Green’



(c) Option Q -table for ‘Goto Yellow’



(d) Option Q -table for ‘Goto Blue’

Figure 11: Option Q -table for each of the provided options learnt with Intra-option Framework

5.2.4 Q-Table for all the states

The following plot captures the (Intra-option Global) Q-table over all primitive actions and options for all five distinct passenger locations and all four distinct destination locations.

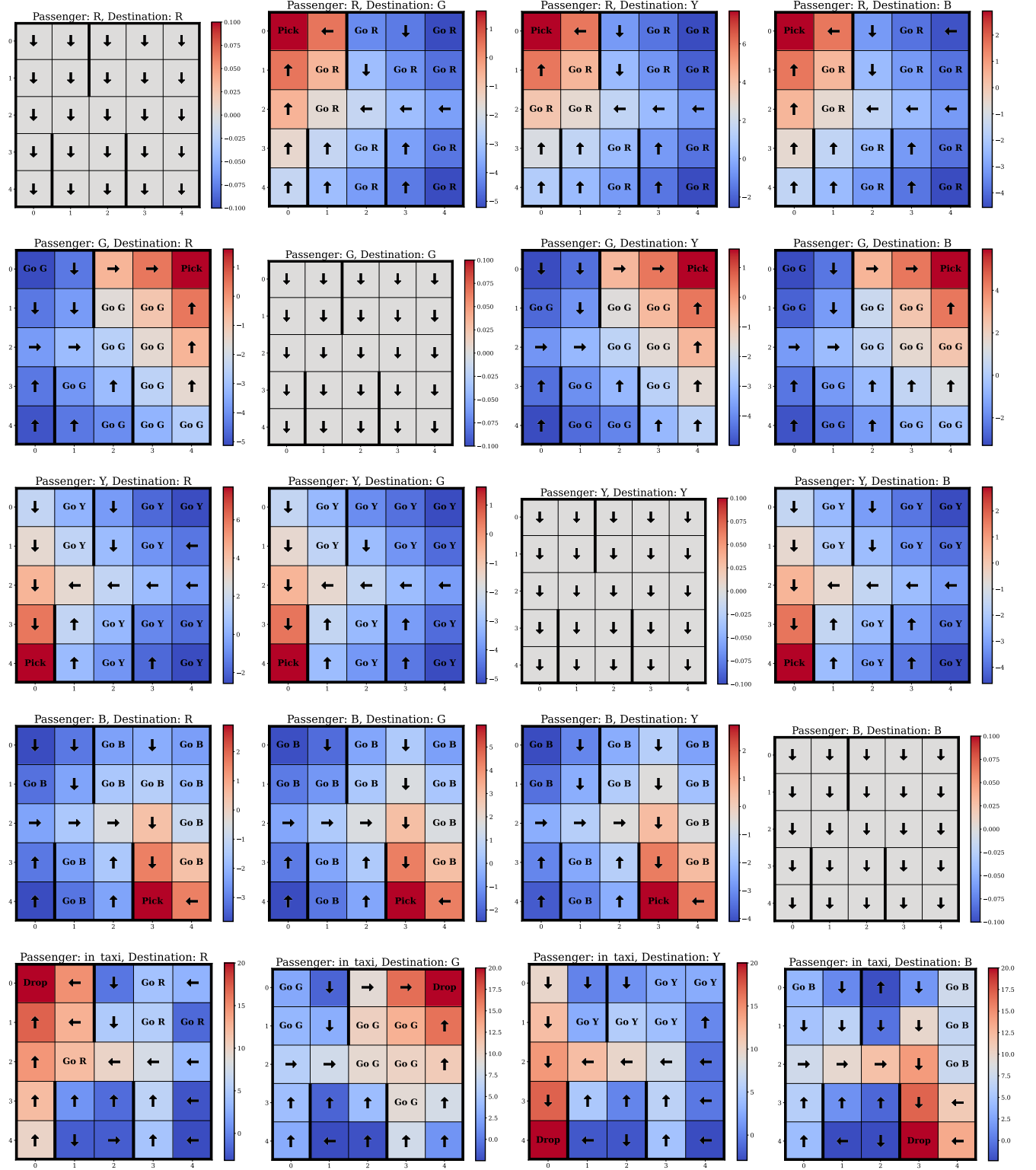


Figure 12: The Q-Table learnt through Intra-option for 20 distinct environment scenarios

5.3 Inference

This section provides some inference on the reward curve (and the Q -value learnt) through each of the SMDP framework and the Intra-option Q -Learning and compares them with each other and with the reward curve obtained through the conventional Q -learning approach of learning with only primitive actions without options.

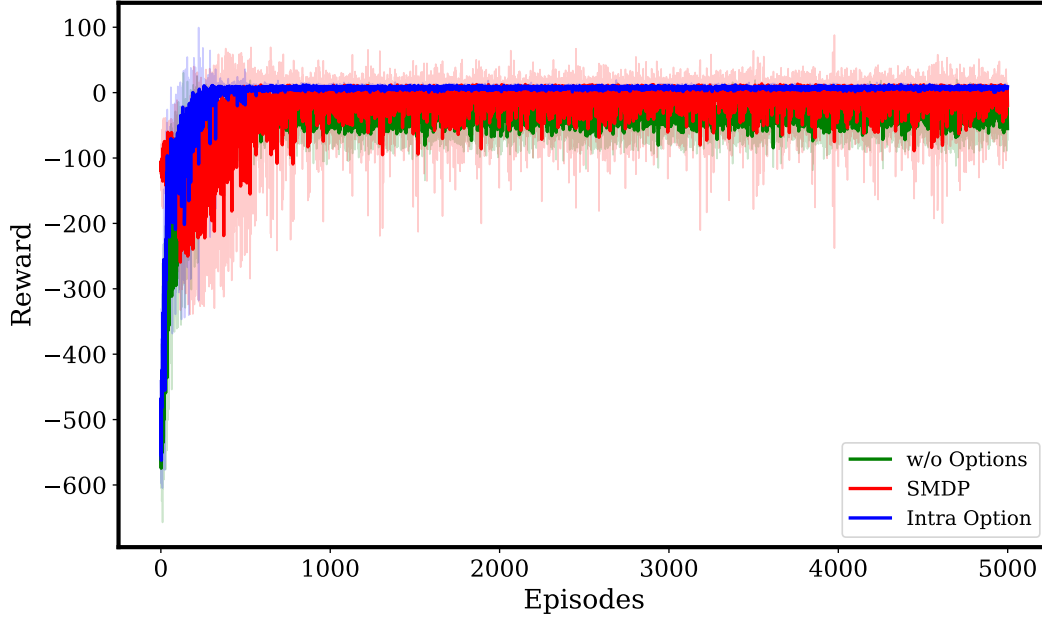


Figure 13: Reward vs Episodes (SMDP vs Intra option vs conventional approach (w/ o options)): Averaged over 5 runs without a moving window

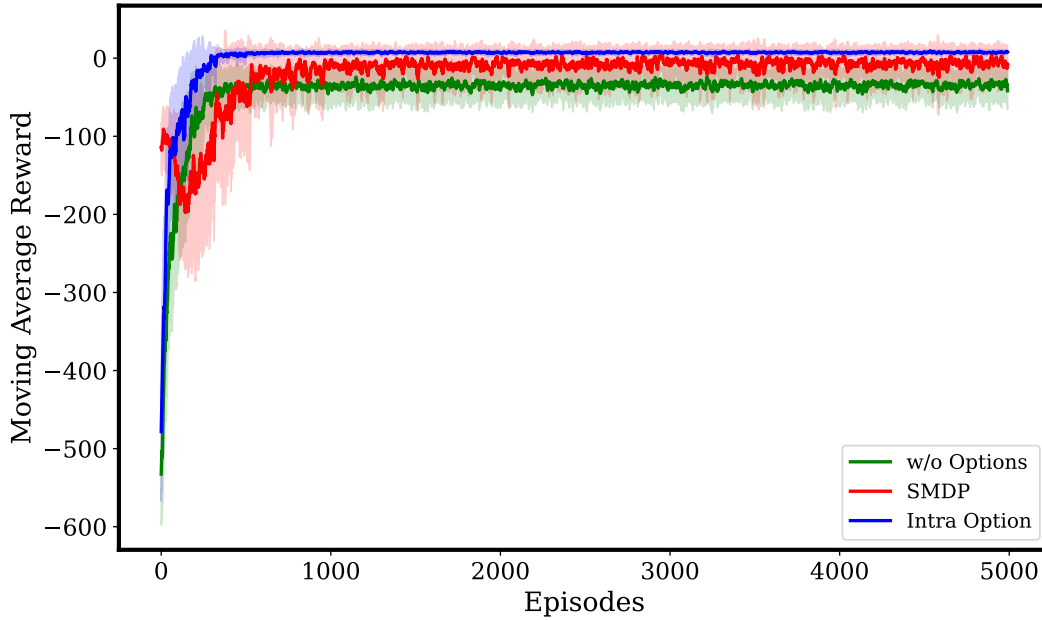


Figure 14: Reward vs Episodes (SMDP vs Intra option vs conventional approach (w/ o options)): Averaged over 5 runs with a moving window of size 10

5.3.1 Reward

In this section, we compare the SMDP, the intra option framework and the conventional Q-Learning from the reward vs episode point of view.

5.3.1.1 Observations

: Here are some observations:

1. Clearly, the Intra option converges faster to a better total reward (converges $\sim +7.86$) as compared to SMDP (converges to ~ -7.4) as compared to conventional Q-Learning (converges to ~ -41.8). The reader may note that for any starting state the optimal reward achievable lies between 4 to 16 depending upon the initial state the environment begins with).
2. The policy learnt with the Intra option converges faster. In the Reward vs Episode plot, we observe a spike in the SMDP learning (at around episode 20) then it comes down (and achieved minimum at around episode 150) then it again picks up, and toward the end, it converges to a better total reward as compared to the traditional Q learning without options.

5.3.1.2 Possible reasons

The observed behaviour, where intra-option Q-learning converges faster to a better total reward compared to SMDP Q-learning, which in turn converges faster than conventional Q-learning, can be explained by the advantages and characteristics of each approach.

1. Intra-Option Q-learning Advantages:

- *Temporal Abstraction:* Intra-option Q-learning allows for the learning of policies at multiple levels of temporal abstraction in this case learning to reach the four colored states, which means the agent can make decisions based on higher-level options rather than just individual primitive actions. This abstraction can lead to more efficient exploration and learning, especially in environments with complex state spaces like the Taxi environment.
- *Guided Exploration:* Options provide a form of guided exploration, where the agent can choose higher-level actions that guide its behavior towards specific sub-goals or states of interest. This can help the agent avoid long sequences of primitive actions that might not lead to meaningful outcomes.
- *Reduced Search Space:* By considering options, the agent effectively reduces the search space it needs to explore. Instead of considering every possible primitive action at each time step, the agent can focus on higher-level options, leading to faster convergence.
- *Learning from analogous experience than just immediate action:* Intra option updates the Q value of all the (s, a) pair in all the options that could have taken that particular step at the state. Thus, a single observation is used for multiple experience updates simultaneously learning each subactions as effectively as possible, leading to faster convergence.

2. SMDP Q-learning Characteristics:

- *Variable Duration Actions:* SMDP Q-learning deals with actions that have variable durations, which introduces an additional level of complexity compared to traditional Q-learning. While this can potentially lead to more flexible and realistic behaviour, it also requires the agent to learn not only action values but also the durations associated with each action. This additional complexity might slow down convergence initially.
- *Complexity of Options:* While SMDP Q-learning introduces options as well, these options might not be as well-structured (from the SMDP view point, as the agent may travel through the entire grid world to terminate the option) or as beneficial as those learned through intra-option Q-learning.

3. **Conventional Q-learning Disadvantages:** Lack of Temporal Abstraction: Traditional Q-learning operates at the level of primitive actions only, which can lead to slower convergence, especially in complex environments. The agent needs to explore and learn the value of each individual action in every possible state, which can be computationally expensive and inefficient.

The observed spike in SMDP Q-learning followed by fluctuations and eventual convergence to a better total reward compared to conventional Q-learning can be attributed to the initial exploration of the variable duration actions and the complexity associated with learning these actions. Once the agent starts to understand the dynamics of the environment and learns the durations associated with each action, it can gradually improve its policy and converge to a better total reward.

In summary, intra-option Q-learning benefits from temporal abstraction and guided exploration, leading to faster convergence and better total rewards compared to both SMDP Q-learning and conventional Q-learning in the Taxi environment. SMDP Q-learning, while initially slower due to the complexity of variable duration actions, eventually converges to a better total reward compared to conventional Q-learning, showcasing the benefits of considering temporal abstraction in RL tasks.

5.3.2 Learnt Q Table, aka learnt policy

In this section, we compare the SMDP and the intra-option framework considering the final Q-table generated, aka the final policy learnt.

5.3.2.1 Observation

Here are some observations.

1. The Q-Table disregards states where the passenger and destination overlap, deeming them completed and exempt from further learning.
2. In the first 16 stages, the objective is reaching the passenger location since the passenger hasn't boarded the taxi yet. Once the passenger is picked up, the state transitions to one of the last 4 states, with the goal changing to reaching the destination location.
3. The Q-values and optimal actions show satisfactory updates for most states, but less-visited states far from the goals may experience slower updates.
4. Despite satisfactory results, the SMDP Q-Tables aren't optimal due to infrequent updates. Only the initial state sees frequent updates, neglecting learning in intermediate states.

5. In contrast, when employing Intra Option methods, the Q-Tables demonstrate robust updates, showcasing anticipated optimal actions. This effectiveness stems from a fundamental difference: In Intra Option learning, Q-values receive updates not only for the initial and completion states but also for all intermediate steps within an option.
6. The distinct advantage of Intra Option learning lies in its comprehensive approach to updating Q-values. Unlike in other methods, where updates primarily occur at the start and end of an option, Intra Option learning ensures that Q-values are adjusted for every step taken within an option. This includes updates for all possible options, including primitive actions, that could have been chosen at each intermediate step.
7. By updating Q-values at every stage of an option, Intra Option methods provide a more nuanced understanding of the environment's dynamics. This granularity enables the agent to learn not only the optimal actions to take but also the most effective sequences of actions to achieve a goal.
8. Consequently, Intra Option learning tends to yield superior performance compared to other methods, especially in tasks with complex action sequences or when navigating environments with long horizons. The ability to update Q-values at each step enhances the agent's adaptability and facilitates more efficient exploration of the state-action space.

6 Alternate Options

In this section, we introduce and explore options that are distinct from the provided options and compare their performances.

6.1 Deterministic Options

The paper entitled ‘Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning’ [2] that forms the foundation for this assignment completely focuses on deterministic options as compared to stochastic options. Followingly, we introduce four deterministic options inspired by the provided options.

6.1.1 The options

We introduce four deterministic options, that are — ‘Goto Red’, ‘Goto Green’, ‘Goto Yellow’ and ‘Goto Blue’. The Option initiation state and the termination probability is precisely the same as that given in the Provided Option. The only difference is the policy of these options that are deterministic. At any state, the action taken is the arg max of the earlier learnt Q-table, that is basically the optimal action. The goal here is to see, how well the frameworks behave and how fast they converge.

6.1.2 Plots

This section attaches some plots pertaining to the deterministic policy.

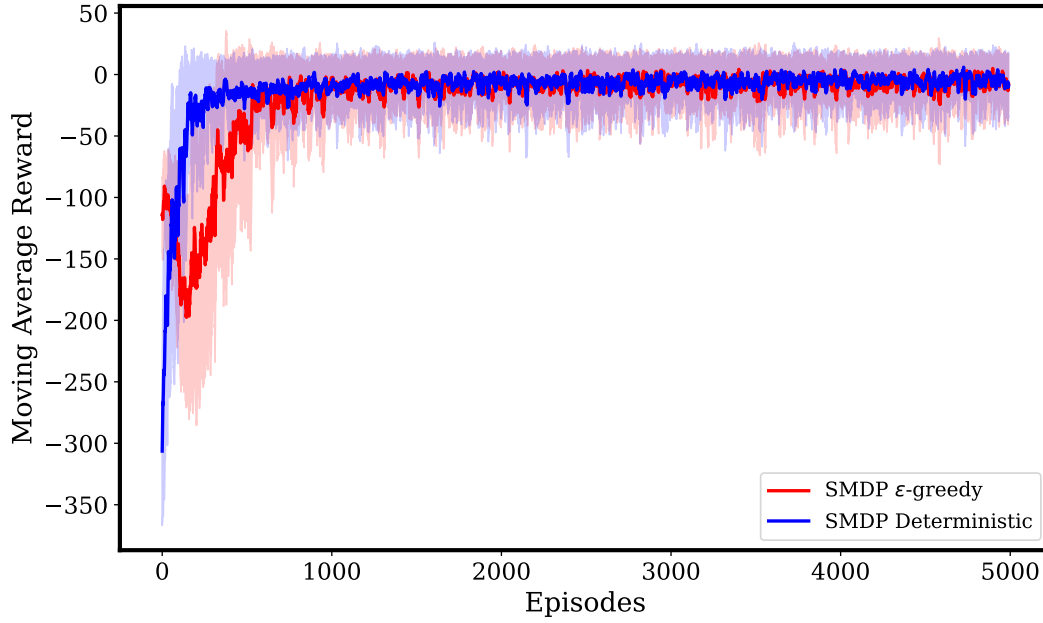


Figure 15: Reward vs Episodes (averaged over 5 runs with a moving window of size 10):
SMDP — ϵ -greedy vs a deterministic policy

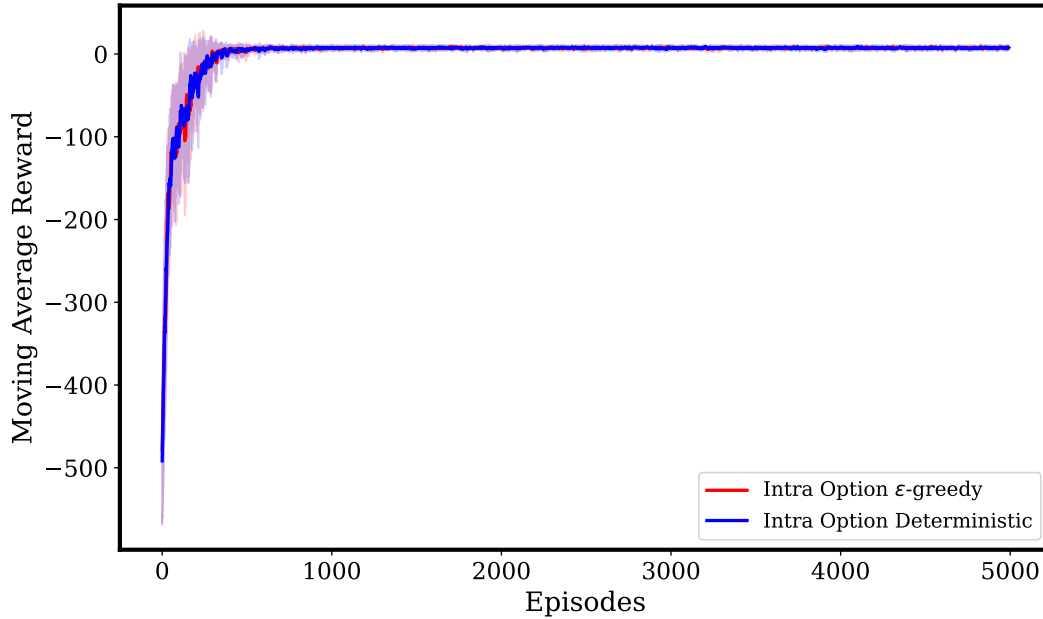


Figure 16: Reward vs Episodes (averaged over 5 runs with a moving window of size 10):
Intra-option — ϵ -greedy vs a deterministic policy

6.1.3 Observation and Inference

We deduce the following inference based on the observations drawn from the definitions of the options defined and the plots.

1. Clearly, the variance obtained in the reward vs episode plot in the deterministic policy is significantly less as compared to that of the ϵ -greedy.

2. It converged to the same optimal policy to which the the ϵ -greedy policy converged.
3. It converged to the optimal policy much faster than the ϵ -greedy.
4. Here the relative behaviour between SMDP and Intra option is analogous to that we observed in the earlier plots. But the reader may not that SMDP also converged faster without any dip as that in the ϵ -greedy.

6.2 Goto two bottleneck cells D and E

Earlier in the problem, we have four distinct options to go to the four colored cells, one of which is a potential termination state. Thus, learning the options makes learning the path to the termination state faster. If we look carefully at the environment grid-world (Figure 17), we have three long barriers. If an agent wants to reach the Red cell (0, 0) or the Yellow cell (4, 0) either it can reach directly if it is at the nearby cells, or it has to use the cell (2, 1) to arrive at any of that two coloured cells. Likewise, if the agent wants to reach the Green (0, 4) or the Blue cell (4, 3), it may reach directly if it is at some nearby cell, or it has to reach the cell (2, 3), before it can go to any of those two latter mentioned coloured cells. Thus, both the cells (2, 0) and (2, 3) constitute a set of bottleneck states reaching at which the agent can efficiently reach any of the colored state. Thus, we define two new stochastic options based on this observation to find both of the above bottleneck states. The reader may observe that we are performing the option execution only with the Intra-option framework as the SMDP framework performed significantly poorly (mostly because of issue in hyperparameter tuning).

We denote the cell (2, 0) as cell D and the cell (2, 3) as the cell E .

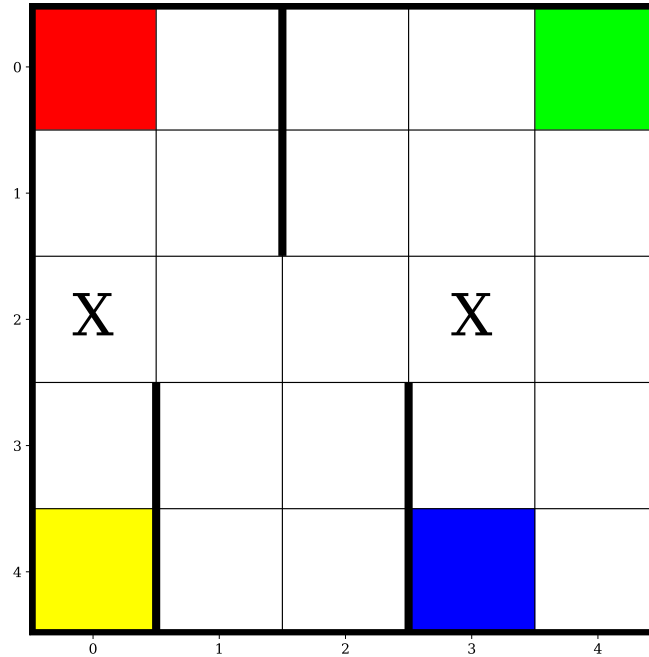


Figure 17: The two bottleneck states $D(2, 0)$ and $E(2, 3)$ marked in X .

6.2.1 The options

These are the newly defined options based on the cell D and E .

6.2.1.1 Goto Cell D(2, 0)

The ‘Goto cell D’ option $o_D := \langle I_D, \pi_D, \beta_D \rangle$ is defined as follows:

- $I_D :=$ The set of all the states in `env.observation_space` where the taxi is not located at the cell D, that is — (2, 0).
- $\pi_D :=$ Choose among the first four primitive actions, that are — move south, move north, move east, move east ϵ -greedily.
- For each state s , the ‘Goto cell D’ termination probability goes as follows:

$$\beta_A(s) = \begin{cases} 1 & \text{if at state } s \text{ taxi is at the cell } D \\ 0 & \text{otherwise} \end{cases}$$

6.2.1.2 Goto Cell B(2, 3)

The ‘Goto cell B’ option $o_E := \langle I_E, \pi_E, \beta_E \rangle$ is defined as follows:

- $I_E :=$ The set of all the states in `env.observation_space` where the taxi is not located at the cell E, that is — (2, 3).
- $\pi_E :=$ Choose among the first four primitive actions, that are — move south, move north, move east, move east ϵ -greedily.
- For each state s , the ‘Goto cell E’ termination probability goes as follows:

$$\beta_E(s) = \begin{cases} 1 & \text{if at state } s \text{ taxi is at the cell } E \\ 0 & \text{otherwise} \end{cases}$$

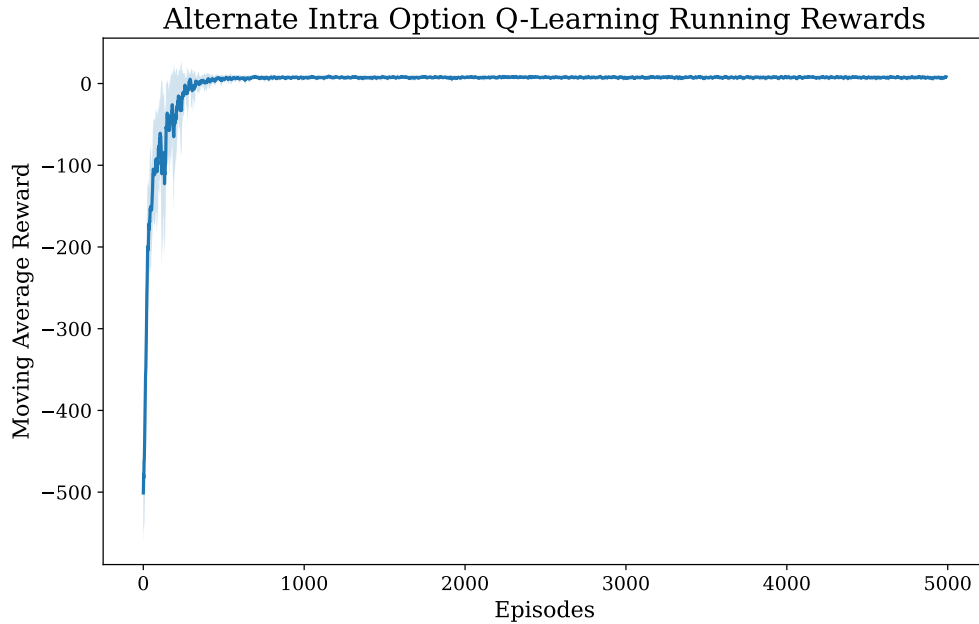


Figure 18: Reward vs Episodes (averaged over 5 runs with a moving window of size 10):
Intra-option: with options ‘Goto Cell D’ and ‘Goto Cell E’

6.2.2 Observation and Inference

Here are the observations made:

1. Clearly, the Intra-option converged to the optimal achievable reward in a smoother and faster way.
2. The Q-table has also converged to the optimal policy within the option.
3. Thus, Options terminating at bottleneck states play a pivotal role in efficiently learning the optimal policy. By focusing exploration on critical decision points within the environment, these options streamline the learning process by directing attention to key areas where decisions have a significant impact on achieving the goal. This targeted exploration reduces overhead and enables the agent to glean valuable insights about the optimal policy more efficiently.

Moreover, the hierarchical nature of options terminating at bottleneck states aligns seamlessly with reinforcement learning frameworks, allowing agents to navigate complex environments with greater precision. By decomposing the task into subgoals represented by bottleneck states, the agent can develop a hierarchical policy that strategically leverages high-level options to efficiently progress towards the ultimate objective. This hierarchical structure not only accelerates learning but also enhances sample efficiency and facilitates generalization and transfer learning across different tasks or environments.

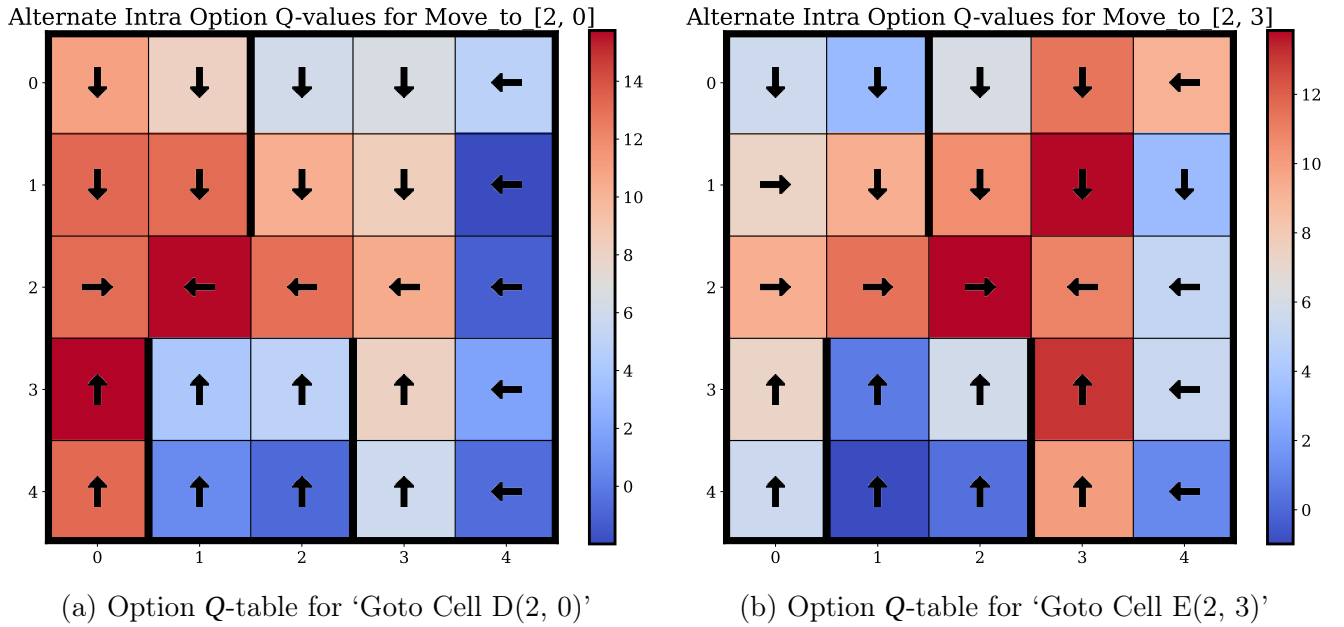


Figure 19: Option Q-table for each of the alternate options (Goto Bottleneck cells) learnt with Intra-option Framework

6.3 Goto the center C(2, 2)

In this section, along with the primitive actions, we study and explore only one option: wherever you are come to the center. The reader may observe that we are performing the option execution

only with the Intra-option framework as the SMDP framework performed significantly poorly (mostly because of the issue in hyperparameter tuning).

The ‘Goto the center $C(2, 2)$ ’ option $o_C := \langle I_C, \pi_C, \beta_C \rangle$ is defined as follows:

- $I_C :=$ The set of all the states in `env.observation_space` where the taxi is not located at the cell C , that is $— (2, 2)$.
- $\pi_C :=$ Choose among the first four primitive actions, that are $—$ move south, move north, move east, move east ϵ -greedily.
- For each state s , the ‘Goto the center C ’ termination probability goes as follows:

$$\beta_C(s) = \begin{cases} 1 & \text{if at state } s \text{ taxi is at the cell } C \\ 0 & \text{otherwise} \end{cases}$$

Option termination state at $[2,2]$. So, 6 primitive actions and 1 option.

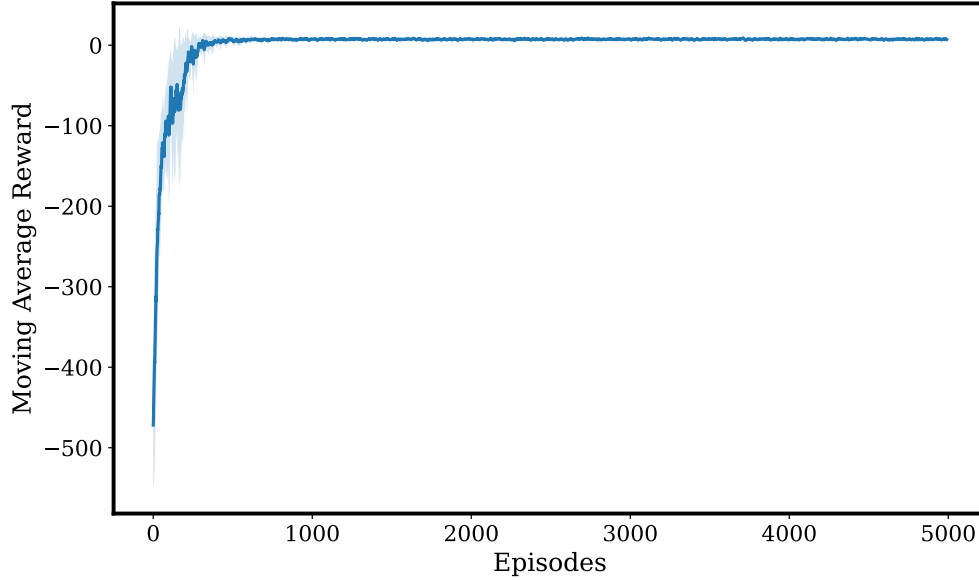


Figure 20: Reward vs Episodes (averaged over 5 runs with a moving window of size 10):
Intra-option: with options ‘Goto the center C ’

Alternate Intra Option Q-values for Move to [2, 2]

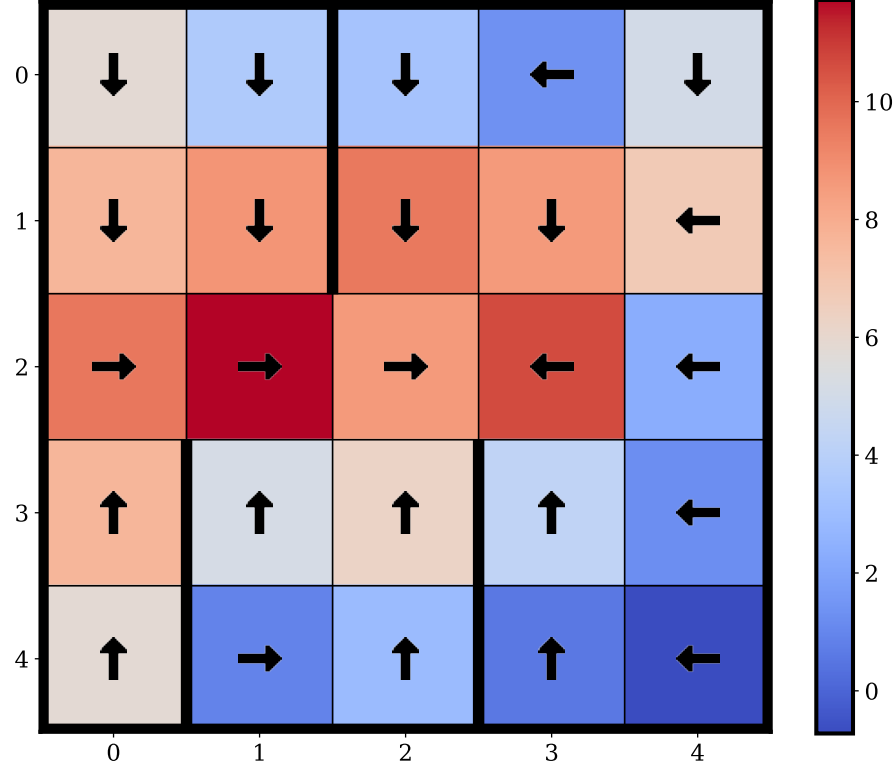


Figure 21: Option Q-table for the alternate options (Goto the center) learnt with Intra-option Framework

6.3.1 Observation and Inference

Here are the observations made:

1. Clearly, go to the center is a one-option approximation of the two-options framework (going to the bottleneck stated) described above.
2. From the Q table, we may conclude that the agent has learnt the option effectively.
3. From the reward vs episode curve, we conclude that it has converged to the optimal policy (analogous to the two-options framework described above).

7 Conclusion

In this assignment, we studied and implemented the SMDP and Intra option Q-Learning, and compared them with the Conventional Q-Learning (without options).

8 References

- [1] Thomas G. Dietterich. Hierarchical reinforcement learning with the maxq value function decomposition. *arXiv preprint cs/9905014*, 1999.

- [2] Richard S. Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Journal of Artificial Intelligence Research*, 1998.